Slide 1 - Introduction to Programming

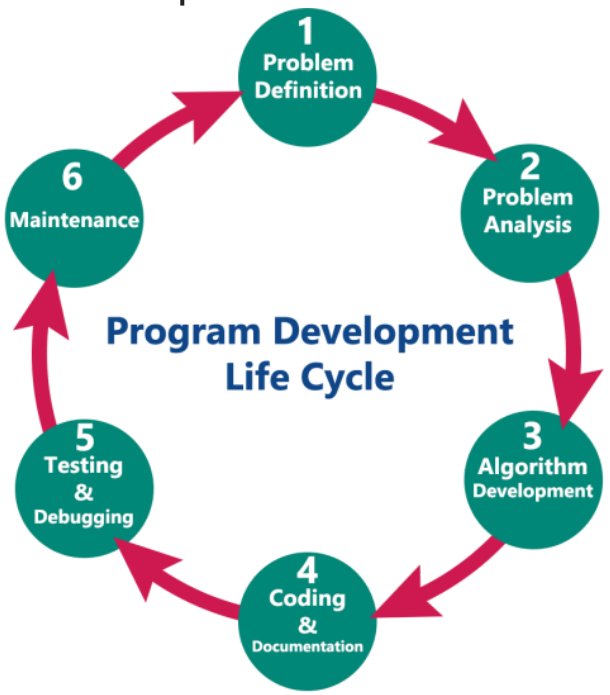


Slide 2 - OUTLINE

- Introduction
- Software vs Hardware in Program Execution
- Problem Solving Phase
- Pseudocode and Algorithm
- The Flowchart
- Flowchart Basic Symbols
- Examples Set I
- Basic Programming Concepts
- Examples Set II
- Loops and Arrays

Slide 3 - INTRODUCTION

- A typical programming task can be divided into the following phases:
- Problem solving: write the steps needed to solve the problem. These steps are called an algorithm.
- Implementation: write the program in a programming language such as Java, Python, or JavaScript.



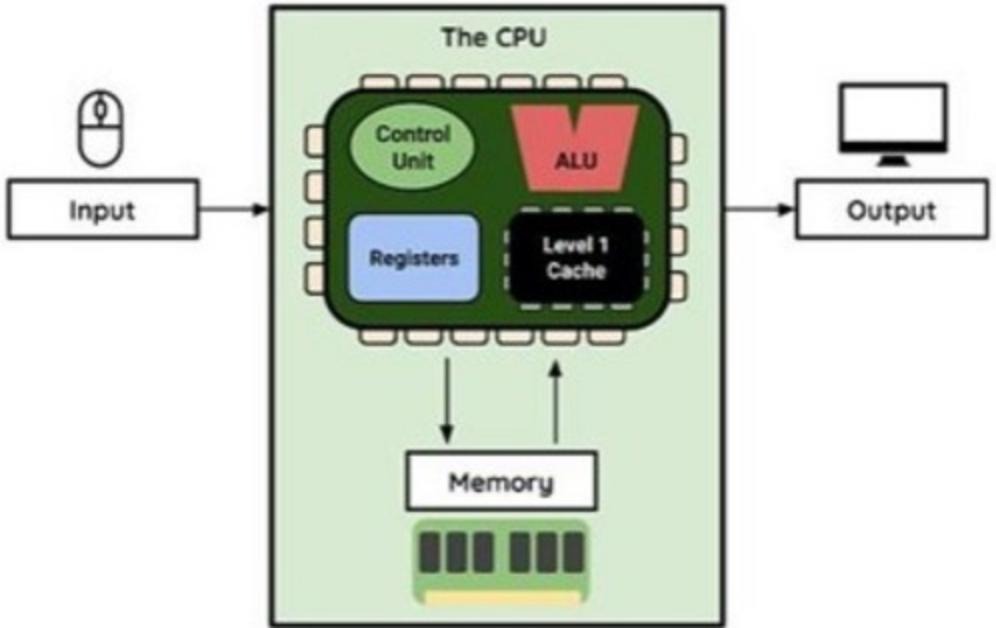
Slide 4 - SOFTWARE VS HARDWARE IN PROGRAM EXECUTION

- Software is the set of instructions written by the programmer.
- Hardware is the physical machine that carries out those instructions.
- The software tells the computer what to do.
- The hardware performs the actual work, such as processing, storing data, and handling input/output.

```
Software: program instructions
         ↓
Hardware: CPU, memory, storage, input/output devices
         ↓
Program execution
```

Slide 5 - HOW THE CPU WORKS

- The CPU (Central Processing Unit) is the brain of the computer.
- It carries out the instructions of a computer program.
- The CPU is composed of:
  - Control Unit (CU): fetches instructions from memory, decodes them, and sends them to the correct part of the CPU.
  - Arithmetic and Logic Unit (ALU): performs calculations and logic operations such as addition, comparison, and decision-making.



Slide 6 - PROBLEM SOLVING PHASE

- First, write a general algorithm to describe the solution.
- A pseudocode is an informal, language-like way to plan an algorithm.
- It is easy to read and helps the programmer think clearly.
- Then, refine the algorithm until it is detailed enough to be implemented in a programming language.

Slide 7 - PSEUDOCODE & ALGORITHM

Write an algorithm and draw a flowchart to determine the student's final grade and indicate whether it is passing or failing. The final grade is calculated as the average of four marks.

PSEUDOCODE

```
Input a set of 4 marks.
Calculate the average of these marks by dividing their sum by 4.
If the average is below 50, print "FAIL"; else print "PASS".
```

ALGORITHM

```
Step 1: Input M1, M2, M3, M4
Step 2: GRADE ← (M1 + M2 + M3 + M4) / 4
Step 3: If (GRADE < 50) then
            print "FAIL"
        else
            print "PASS"
        endif
```

Slide 8 - THE FLOWCHART

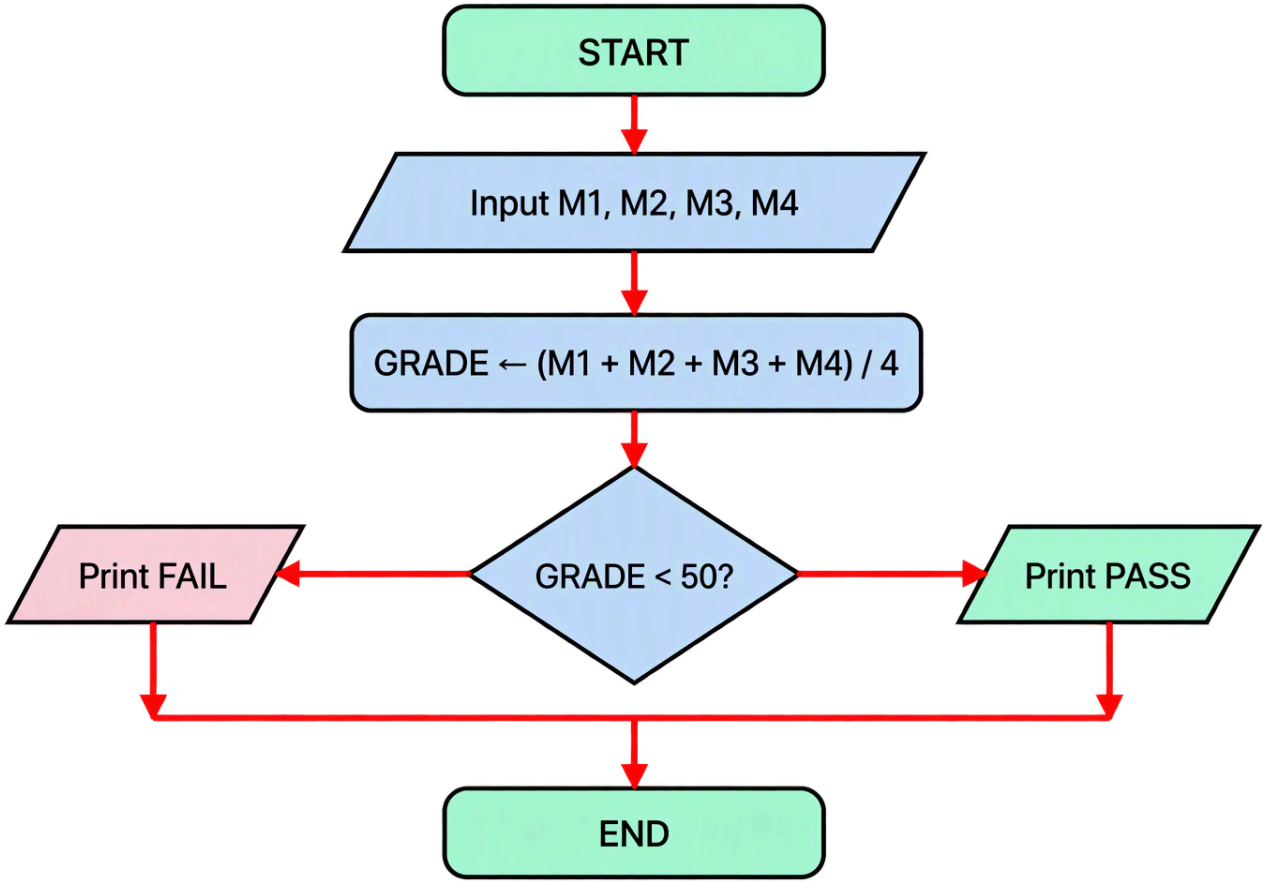
A flowchart

- Shows the steps of an algorithm in order.
- Helps us understand the logic of a program.
- Uses symbols to represent actions, decisions, and flow.
- It is a visual way to plan a solution before coding.

Slide 9 - FLOWCHART BASIC SYMBOLS

Name	Symbol	Use in Flowchart
Oval	○	Denotes the beginning or end of the program
Parallelogram	▱	Denotes an input/output operation
Rectangle	▭	Denotes a process to be carried out, e.g. addition, subtraction, division, etc.
Diamond	◇	Denotes a decision (or branch) to be made. The program should continue along one of two routes (e.g. IF/THEN/ELSE).
Flow line	→	Denotes the direction of logic flow in the program

Slide 10 - FLOWCHART Example 1



Slide 11 - Example 2

Write an algorithm and draw a flowchart to convert the length from feet to centimeters. The length in centimeters is equal to 30.48 times the length in feet.

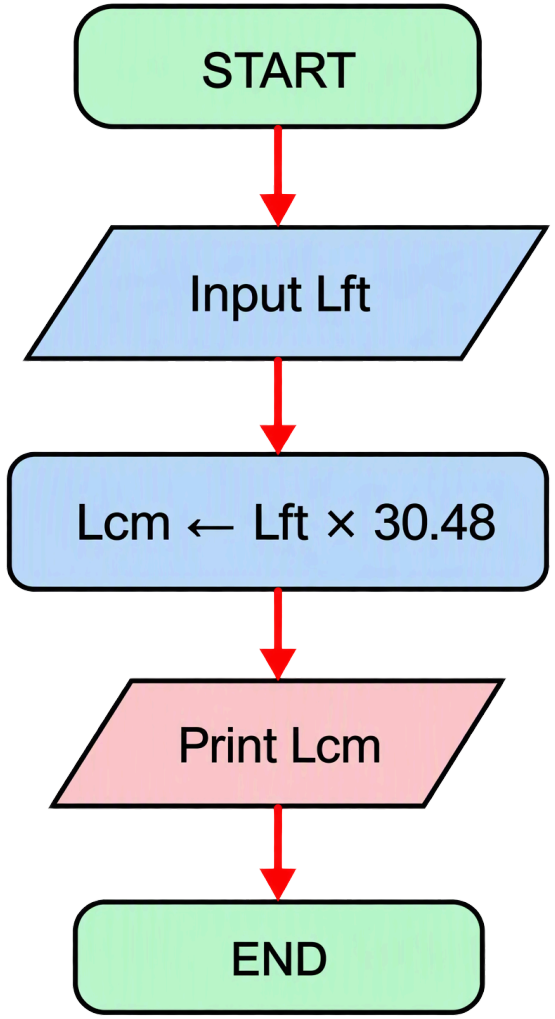
PSEUDOCODE

```
Input the length in feet (Lft).
Calculate the length in centimeters (Lcm) by multiplying the length in feet (Lft) by 30.48.
Print the length in centimeters (Lcm).
```

ALGORITHM

```
Step 1: Input Lft
Step 2: Lcm ← Lft × 30.48
Step 3: Print Lcm
```

Slide 12 - FLOWCHART Example 2



Slide 13 - VARIABLES, CONSTANTS, AND ASSIGNMENT

- A variable stores data that can change while a program is running.
- A constant stores a value that remains fixed.
- Assignment means putting a value into a variable.

```
Total ← 0
Score ← 80
PI ← 3.14
Area ← Length × Width
```

Slide 14 - RELATIONAL OPERATORS

- Relational operators are used to test or define some kind of relation between two entities.
- Equal To “=”
- Less Than “<”
- Greater Than “>”
- Less Than Or Equal To “≤”
- Greater Than Or Equal To “≥”
- Not Equal To “≠”

Slide 15 - DECISION STRUCTURES

- A logical expression describes a condition that we want to test.
- The algorithm structure is as follows:

```
If [condition] then
    true alternative
else
    false alternative
endif
```

Slide 16 - EXAMPLE 3

Write an algorithm and draw a flowchart to calculate the roots of a second order equation $ax2 + bx + c = 0$. The roots are calculated as $x1$ = (-b + √Δ)/(2a) and x2 = (-b - √Δ)/(2a) where Δ = $b^2 - 4ac$ is the discriminant.

PSEUDOCODE

```
Input the coefficients a, b, and c of the quadratic equation.
Calculate the discriminant DELTA = b^2 - 4ac.
If DELTA < 0 then
    Print "No real roots"
    Exit
Endif
Calculate the first root x1 = (-b + sqrt(DELTA)) / (2a).
Calculate the second root x2 = (-b - sqrt(DELTA)) / (2a).
Print the roots x1 and x2.
```

ALGORITHM

```
Input the coefficients a, b, and c of the quadratic equation.
DELTA = b^2 - 4ac
If DELTA < 0 then
    Print "No real roots"
Else
    x1 = (-b + sqrt(DELTA)) / (2a)
    x2 = (-b - sqrt(DELTA)) / (2a)
    Print x1 and x2
Endif
```

Slide 17 - EXAMPLE 4

Write an algorithm and draw a flowchart that reads two values, determines the largest value, and prints the largest value with an identifying message.

PSEUDOCODE

- Input the two values VALUE1 and VALUE2.
- If VALUE1 is greater than VALUE2, then the largest value (MAX) is VALUE1; else the largest value (MAX) is VALUE2.
- Print “The largest value is” MAX.

ALGORITHM

```
Step 1: Input VALUE1, VALUE2
Step 2:
If (VALUE1 > VALUE2) then
    MAX ← VALUE1
else
    MAX ← VALUE2
endif
Step 3: Print “The largest value is” MAX
```

Slide 18 - EXAMPLE 5

Examples
Write an algorithm and draw a flowchart that reads an employee name, overtime hours worked, hours absent and determines the bonus payment.

ALGORITHM

```
Step 1: Input NAME, OVERTIME, ABSENT
Step 2:
If (OVERTIME – (2/3)*ABSENT > 40) then
      BONUS ← 50
else if (OVERTIME – (2/3)*ABSENT > 30)
          then BONUS ← 40
     else if (OVERTIME – (2/3)*ABSENT > 20) then
                BONUS ← 30
          else if (OVERTIME – (2/3)*ABSENT > 10) then
                     BONUS ← 20
               else
                     BONUS ← 10
               endif
          endif
      endif
endif
Step 3: Print “Bonus for” NAME “ is $” BONUS
```

Slide 19 - EXAMPLE 6

Write an algorithm and draw a flowchart that reads three numbers and prints the value of the largest number with an identifying message.

ALGORITHM

```
Step 1: Input N1, N2, N3
Step 2:
If (N1 > N2) then
    If (N1 > N3) then
        MAX ← N1
    else
        MAX ← N3
    endif
else
    If (N2 > N3) then
        MAX ← N2
    else
        MAX ← N3
    endif
endif
```

Slide 20 - LOOPS AND ARRAYS

- A loop repeats a block of instructions until a condition is met.
- A For loop is used when the number of repetitions is known.
- A While loop continues while a condition is true.
- An array is a collection of values stored in one variable.
- Each value is stored in a position called an index.
- Arrays are useful when many related values must be processed together.
- Example: scores[1], scores[2], scores[3] are three numbers in one array.

```
Scores[1] ← 75
Scores[2] ← 90
Scores[3] ← 80
```

Slide 21 - EXAMPLE 6 (USING ARRAY AND LOOP)

Write an algorithm and draw a flowchart that reads three numbers and prints the value of the largest number with an identifying message using an array.

ALGORITHM

```
Step 1: Input ARRAY[1], ARRAY[2], ARRAY[3]
Step 2: MAX ← ARRAY[1]
Step 3: For i ← 2 to 3 do
            If ARRAY[i] > MAX then
                MAX ← ARRAY[i]
            Endif
        Endfor
Step 4: Print "The largest number is", MAX
```

Slide 22 - PYTHON SOLUTION :

- DEFINE THE ARRAY

```
numbers = [12, 45, 7]
max_value = numbers[0]

for value in numbers[1:]:
    if value > max_value:
        max_value = value

print("The largest number is", max_value)
```

- READ ARRAY AS INPUT

```
numbers = []
for i in range(3):
    value = int(input("Enter a number: "))
    numbers.append(value)

max_value = numbers[0]
for value in numbers:
    if value > max_value:
        max_value = value

print("The largest number is", max_value)
```

Slide 23 - JAVASCRIPT SOLUTION 1

- DEFINE THE ARRAY

```
let numbers = [12, 45, 7];
let maxValue = numbers[0];

for (let i = 1; i < numbers.length; i++) {
  if (numbers[i] > maxValue) {
    maxValue = numbers[i];
  }
}
console.log("The largest number is", maxValue);
```

- READ ARRAY AS INPUT

```
let numbers = [];
for (let i = 0; i < 3; i++) {
  let value = Number(prompt("Enter a number: "));
  numbers.push(value);
}
let maxValue = numbers[0];
for (let i = 1; i < numbers.length; i++) {
  if (numbers[i] > maxValue) {
    maxValue = numbers[i];
  }
}
console.log("The largest number is", maxValue);
```

Slide 24 - JAVASCRIPT SOLUTION 2: USE forEach

```
let numbers = [12, 45, 7];
let maxValue = numbers[0];

numbers.forEach((value) => {
  if (value > maxValue) {
    maxValue = value;
  }
});

console.log("The largest number is", maxValue);
```